TECHNICAL GUIDE · .NET TEAM

RoslynGate

How an AI agent understands .NET with Roslyn, without turning its context into a soup of text.

Architecture, build pipeline, semantic reference resolution, MCP contract, measured performance, and real limits. This document explains where RoslynGate connects to MSBuild and Roslyn, what the AI does with each response, and what can be scaled next.

CONTENTS

What you will find here

01	The problem: text is not semantics	03
02	Where RoslynGate enters the .NET pipeline	04
03	The journey of a query	05
04	How a reference is known	06
05	Internal architecture	07
06	How an AI agent uses it	08
07	Performance and token economics	09
08	Limits, freshness, and the SDK	10
09	Scalability and expansion	11

One-sentence summary

RoslynGate turns questions about .NET symbols into semantic queries against a solution loaded by MSBuild, and returns compact JSON with file and line so an agent can decide without reading half the repository.

4–11 ms
warm queries

99
projects in the legacy test

21–34%
fewer tokens in real A/B

~27 s
initial workspace load

> Token figures come from two queries measured with Claude Code on a real solution; they are not a universal percentage. Latency is the price of loading MSBuild once per process.

01 — THE PROBLEM

Text is not semantics

An agent that searches for symbols with `grep` finds letters. A compiler finds meaning. The difference matters when there are overloads, partial classes, generics, generated code, or projects referencing each other.

1 False positives

The word “Doors” appears in comments, strings, documentation, and names of other types. Text search mixes them with real references.

2 False negatives

Semantic references may live in generated files or resolve through an interface. Text does not always connect them.

3 Overloads

`Open()`, `Open(string)`, and `Open(int)` share a name but not a symbol. The agent must disambiguate.

4 Expensive context

Dumping entire files consumes tokens and pushes useful information out. The goal is the right location, not the whole file.

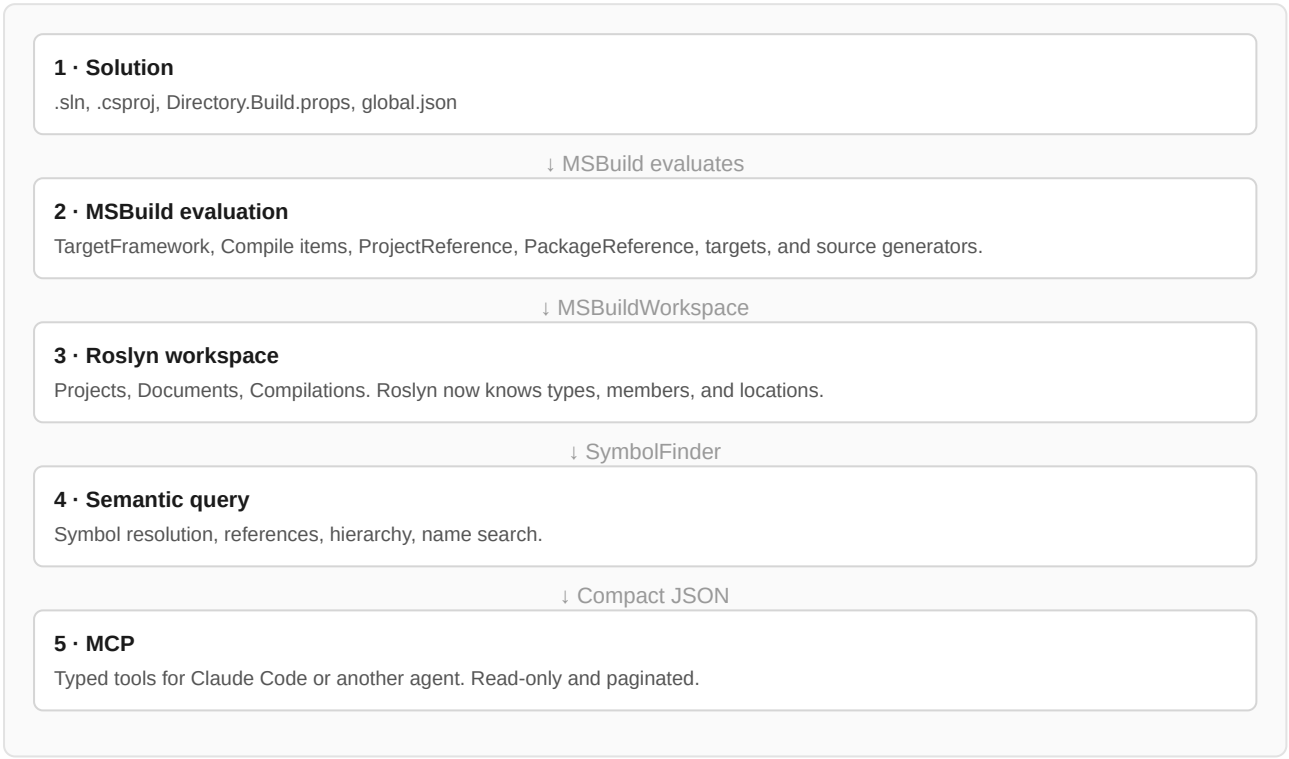
> *RoslynGate does not replace grep for free text. It complements it when the question is “what symbol is this, and who uses it?”*

The tool is read-only: it observes the solution; it does not edit code or run tests.

02 — WHERE IT ENTERS

The .NET pipeline, stage by stage

RoslynGate does not inject itself into the compiler. It sits beside it: MSBuild prepares the same information a build would use, and Roslyn queries that information.



What each stage provides

STAGE	OWNER	WHAT IT RESOLVES
Evaluation	MSBuild / SDK	Which files and references truly belong to the project.
Compilation	Roslyn	Types, members, overloads, scopes, and semantic model.
Query	RoslynGate	Ordering, deduplication, pagination, and agent-friendly format.

03 — A QUERY

The journey of a question

The agent does not see a .NET solution. It sees MCP tools. RoslynGate translates between the two worlds.

```
Claude Code
  ↓ tools/call: find_symbol_references
RoslynGate MCP
  ↓ validates arguments
SymbolResolver
  ↓ name or file + line
Roslyn Workspace
  ↓ SymbolFinder.FindReferencesAsync
Semantic result
  ↓ order, dedupe, offset, limit
Compact JSON
  ↓ refs: [file, line, kind]
Agent
```

Example response

```
{
  "q": "ViewModels.DatabaseAccess.Doors",
  "definition": {
    "n": "Doors",
    "k": "class",
    "f": "ViewModels/DatabaseAccess/Doors.cs",
    "l": 16
  },
  "total": 167,
  "offset": 0,
  "truncated": true,
  "nextOffset": 200,
  "refs": [
    ["ViewModels/DatabaseAccess/Doors.cs", 22, "read"],
    ["Services.cs", 1820, "read"]
  ]
}
```

If the page is truncated, the agent receives `nextOffset` and a `hint`. It does not have to calculate the next page or guess.

How a reference is known

A reference is not found by text: Roslyn's semantic model is asked which symbol sits at each syntax node.

1 Resolve the symbol

By qualified name or by file + line. If there are multiple candidates, RoslynGate returns candidates instead of guessing.

2 FindReferencesAsync

SymbolFinder walks the compilation and returns locations pointing to the same symbol, including referenced projects.

3 Classify

Each use is labeled as declaration, call, inheritance, read, or implicit.

4 Order and paginate

File, line, column, and kind with stable ordering. The result is deterministic and bounded.

Cases text cannot distinguish

CASE	WHAT ROSLYN DOES
Overloads	Distinguishes signature and parameter types.
Partial classes	Joins parts into the same symbol.
Interfaces	Distinguishes implementation from concrete call.
Generated code	Can include documents generated by Razor or source generators.
Comments / strings	Not semantic references and not counted.

05 — ARCHITECTURE

Three layers, one direction

The design separates protocol, queries, and composition. Each layer has one responsibility and does not skip the others.

Cli

argv, logging, exit codes, `--check`, host composition. It does not talk to Roslyn directly.

Mcp

MCP tools, compact JSON, actionable errors, pagination, instructions. It does not reference Roslyn.

Engine

Solution, MSBuildWorkspace, SymbolResolver, SymbolFinder, hierarchy, search. It does not know MCP.

The snapshot

RoslynGate does not keep an on-disk index. It keeps an in-memory snapshot of the loaded solution. Queries reuse compilations and symbols; the MCP process lives for the agent session.

> *The first query pays for MSBuild. The next ones live off the snapshot. That is the difference between “it takes 27 seconds” and “it answers in milliseconds”.*

Tool contract

TOOL	USE
`get_symbol_definition`	Resolve one symbol by name or file + line.
`find_symbol_references`	Paginated semantic references; `includeDefinition` avoids an extra call.
`get_type_hierarchy`	Base types, interfaces, derived types, implementations.
`search_symbols`	Search by fragment when the full name is unknown.
`reload_workspace`	Reload after project changes; the agent uses it when it sees `stale`.

06 — AGENTS

How an AI agent uses it

The agent does not “run Roslyn”. It calls MCP tools. The magic is turning an ambiguous question into a semantic query and returning only what is needed.

1 Define

The agent asks for the definition of an exact name. If ambiguous, it receives candidates and chooses or asks for precision.

2 Reference

It asks for semantic uses. It receives file, line, and kind, not entire files.

3 Paginate

If `truncated=true`, it uses `nextOffset` and repeats. It never invents the next page.

4 Answer

It returns `file:line` to the user, knowing the symbol was resolved by Roslyn.

The skill and instructions

A Claude Code skill can say: “for an exact name, call `find_symbol_references` with `includeDefinition=true`; do not call `get_symbol_definition` too”. That cuts turns and repeated context.

```
---
name: roslyn-symbols
description: Resolve .NET symbols with RoslynGate before Grep.
allowed-tools: mcp__roslyngate__get_symbol_definition mcp__roslyngate__find_symbol_references
---
For an exact name, call find_symbol_references with includeDefinition=true.
If truncated, call again with offset=nextOffset.
```

> A skill does not measure savings by itself. That needs an A/B: same prompt with and without RoslynGate, comparing real tokens and cost.

07 — PERFORMANCE

Tokens, latency, and the price of MSBuild

The tool wins when it stops the agent from reading entire files. The cost is an initial MSBuild load and more turns if the agent does not use the tools well.

METRIC	WITH ROSLYNGATE	WITHOUT ROSLYNGATE	DIFFERENCE
3 symbols · tokens	94,158	142,506	33.9% fewer
3 symbols · cost	$0.07	$0.10	30% lower
6 symbols · tokens	103,904	131,865	21.2% fewer
6 symbols · cost	$0.14	$0.18	22% lower
Initial load	~27 s	—	once
Warm queries	4–11 ms	~10 ms	comparable

How to read the numbers

- Tokens include cache reads; cost comes from Claude Code's `total_cost_usd`.
- RoslynGate latency is higher because each new process loads the workspace; an interactive session keeps it alive.
- In the six-symbol query RoslynGate returned 167 references and the baseline 157: it was not truncation.
- These figures are a sample, not a universal promise.

> The benchmark uses `--no-prewarm` to measure the worst case. In real use, prewarm starts when the session opens and the first query may feel less expensive.

08 — LIMITS

What RoslynGate does not promise

An honest technical document includes what the tool does not do. These limits also explain why the design is read-only and why the SDK is still required.

1 SDK required

The runtime is bundled, but MSBuild needs the .NET SDK. RoslynGate honors `global.json` and reports when the required SDK is missing.

2 Snapshot, not index

If a file changes after loading, the snapshot can be stale. The plan is an incremental watcher, `stale`, and explicit reload.

3 No free text

It does not search comments, strings, or documentation as text. For that, grep/rg remains better.

4 No daemon

Each MCP process loads its own workspace. There is no on-disk cache or shared server across sessions.

Freshness: now implemented

The watcher is on by default and updates `.cs` files in memory. Structural changes (`.csproj` , `.razor` , `global.json`) set `stale` . The fifth tool, `reload_workspace` , reloads without restarting the agent.

```
stale: true
changed: ["ViewModels/DatabaseAccess/Doors.cs"]
try: { tool: "reload_workspace", arguments: {} }
```

A `0` with `stale: true` does not mean “no references”. The AI must reload and retry before concluding.

09 — SCALE

What can be built on top

The base already separates layers and exposes an MCP contract. That allows growth without turning the engine into a monolith.

Incremental watcher

Edit a `.cs` and query without restarting. Project changes set `stale`.

Overloads by type

`parameterTypes=["string"]` resolves the signature without the “choose file+line” round trip.

Impact analysis

Before changing an interface, list affected implementations and call sites.

CI and PR review

Use the same engine to verify references and dependencies in a pipeline.

Architecture maps

Project, layer, and dependency graphs from semantic references.

Multiple solutions

Project or solution filters for monorepos, with selective loading.

What I would not do

I would not turn RoslynGate into a free-text search engine or an editor. That breaks the proposition: semantic precision, read-only, and low context. For text, grep; for symbols, Roslyn.

> *The tool is not trying to be an IDE. It is trying to give an agent the minimum semantic answer it needs so it does not read half the repository.*

READ-ONLY · SEMANTIC · MCP · DETERMINISTIC · PAGINATED · SDK-AWARE